

PROJECT DASHBOARD

Technical Requirements Document

Version 1.2.0 | Full Entity Model

Version	1.2.0
Status	Draft
Date	April 9, 2026
Author	S.A.Taylor
Classification	Internal
Supersedes	v1.1.0 — Added ProjectInvoice join table; Task subtasks via parentTaskId

1. Introduction

1.1 Purpose

This Technical Requirements Document defines the complete data model, entity relationships, API surface, authentication strategy, and business rules for the Project Dashboard — a custom-built, full-stack client and project management application built entirely from scratch with no out-of-the-box solutions.

1.2 Scope

- User accounts with role-based access control (Admin / Client) using passwordless email code authentication
- Client CRM with contact info, measurements, social media, and subscriptions
- Project lifecycle management with status tracking and pricing
- Invoice management with line items, payment tracking, and multi-project support
- Task management with subtasks, scoped to projects
- Service catalog — reusable billable service definitions
- Hierarchical project type taxonomy with sewing-specific sub-classifications
- Pattern style tracking for bespoke and custom sewing projects

1.3 What Changed from v1.1.0

✓ **New in v1.2:** Two structural changes from v1.1.0: (1) Invoice now links to Projects via a ProjectInvoice join table instead of a single optional FK, enabling bundled multi-project invoicing. (2) Task now supports subtasks via a self-referencing parentTaskId field, capped at one level of nesting.

Change	Details
Invoice → Projects (M:M)	Removed Invoice.projectId. Added ProjectInvoice join table. An invoice can now span multiple projects (e.g. one invoice for a wedding dress + branding + website package).
Task subtasks	Added Task.parentTaskId (self-referencing FK, nullable). A null value = root task. A non-null value = subtask. Max one level of nesting enforced at API layer.
Cascade updated	Deleting a Project now also removes its ProjectInvoice join records. Invoice records themselves are preserved.

1.4 Glossary

Term	Definition
TRD	Technical Requirements Document
RBAC	Role-Based Access Control
PK	Primary Key — unique row identifier
FK	Foreign Key — reference to another entity PK
Enum	Enumeration — fixed set of allowed string values
Rich Text	HTML or Markdown formatted text rendered in the UI
OTP	One-Time Passcode — the short code emailed to the user for login
Root Task	A task with parentTaskId = null; may have subtasks beneath it
Subtask	A task with a non-null parentTaskId referencing another Task
Muslin	Test garment made from inexpensive fabric to verify a sewing pattern
Line Item	A single row on an invoice representing a service, quantity, and price

2. Architecture Overview

2.1 Recommended Tech Stack

Layer	Recommended	Alternatives
Frontend	React / Next.js	Vue 3, SvelteKit, Nuxt
Backend / API	Node.js + Fastify or Express	Django, Laravel, Rails
Database	PostgreSQL	MySQL 8, SQLite (dev only)
ORM	Prisma	TypeORM, Drizzle, Sequelize
Auth	Passwordless — Email OTP	See Section 3 for full spec
Email Service	Resend or SendGrid	Postmark, AWS SES, Nodemailer

Layer	Recommended	Alternatives
File Storage	Cloudflare R2 (S3-compatible)	AWS S3, Local (dev only)
Deployment	Docker + Railway / Render	VPS, AWS, Fly.io

2.2 Complete Entity Map

11 Entities: User AuthCode Client Project Invoice InvoiceLineItem Service Task
ProjectType PatternStyleDetail PatternStyleType

Join Tables: ProjectProjectType | ProjectInvoice

3. Authentication — Passwordless Email Code

3.1 Overview

The application uses a fully passwordless authentication flow. No password column exists anywhere in the database. Users authenticate by entering their email, receiving a 6-digit one-time code, and submitting that code to receive a JWT access token and refresh token.

Note: No password field exists on any entity. Do not add one. All authentication must flow through the email code path.

3.2 Auth Flow — Step by Step

Step	Actor	Action
1	Browser	User enters email and submits login form.
2	API Server	POST /auth/request-code — finds or creates User by email, generates a cryptographically secure random 6-digit code, hashes it (bcrypt or SHA-256), stores hash in AuthCode table with expiresAt = now + 15 min, invalidates any prior active code for this user, then sends the plain code via transactional email.
3	Email	Transactional email dispatched: subject "Your login code", body contains the 6-digit code and a 15-minute expiry notice.
4	Browser	User retrieves code from email and enters it on the verification screen.
5	API Server	POST /auth/verify-code — looks up AuthCode by userId, checks: (a) not expired, (b) not used, (c) attempts < 5, then compares submitted code against hash. On match: sets usedAt, issues signed JWT access token (15 min TTL) and opaque refresh token stored in httpOnly cookie (7-day TTL).
6	Browser	Stores access token in JavaScript memory only (never localStorage/sessionStorage). Attaches as Authorization: Bearer <token> on all subsequent requests.
7	API Server	POST /auth/refresh — validates refresh token cookie, rotates it (invalidate old, issue new), returns new access token.
8	API Server	DELETE /auth/logout — deletes refresh token from database, clears cookie. Access token expires naturally.

3.3 AuthCode Entity

3.3.1 AuthCode Schema

Field	Type	Required	Notes / Constraints
id	UUID / Int PK	Yes	Auto-generated primary key
userId	UUID / Int FK	Yes	References User.id
codeHash	String	Yes	bcrypt or SHA-256 hash of the plain code. Raw code never stored.
expiresAt	DateTime	Yes	now + 15 minutes at creation.
usedAt	DateTime	No	Null until consumed. Set on successful verification. Prevents replay.
attempts	Integer	Yes	Default 0. Incremented on each failed verify attempt. Invalidate at 5.
ipAddress	String	No	Requester IP for abuse detection and audit logging.
createdAt	DateTime	Yes	Auto-set on creation

3.3.2 AuthCode Business Rules

- Only one active (unused, non-expired) AuthCode per user at a time. Creating a new code must invalidate any existing active code.
- Raw code is never stored — only its hash. Never log the raw code.
- After 5 failed verify attempts on a single code, immediately invalidate that code.
- Rate-limit POST /auth/request-code: max 5 requests per email address per 10 minutes.
- Set usedAt immediately on successful verification to prevent replay attacks.
- Purge expired and used AuthCode records older than 24 hours via a scheduled cleanup job.

3.4 Token Strategy

Token Type	Details
Access Token (JWT)	Signed HS256 or RS256. Payload: { sub: userId, role, iat, exp }. TTL: 15 minutes. Stored in JS memory on client only.
Refresh Token	Opaque 32-byte random hex string. Stored in DB with userId and expiry. Delivered via httpOnly Secure SameSite=Strict cookie. TTL: 7 days.
Token Rotation	Every /auth/refresh call invalidates the old refresh token and issues a new one. Detected reuse of a revoked token invalidates ALL sessions for that user.
Logout	DELETE /auth/logout deletes the refresh token DB record and clears the cookie. Access token expires within 15 minutes naturally.

4. Entity Definitions

4.1 User

Represents an authenticated account. Admins manage the system; client-role users have read-only access to their own data. No password is stored.

4.1.1 User Schema

Field	Type	Required	Notes / Constraints
id	UUID / Int PK	Yes	Auto-generated primary key
email	String	Yes	Unique. Max 255 chars. The only credential identifier.
role	Enum	Yes	Values: admin client. Default: client.
lastLoginAt	DateTime	No	Updated on each successful code verification.
createdAt	DateTime	Yes	Auto-set on creation
updatedAt	DateTime	Yes	Auto-set on update
deletedAt	DateTime	No	Soft delete. Null = active. Soft-deleted users cannot authenticate.

4.1.2 User Business Rules

- email is unique and is the sole login identifier.
- Role changes may only be made by an admin-role user.
- Deletion is soft (deletedAt). Soft-deleted users cannot request or verify codes.

4.2 Client

Stores all CRM data for individuals or companies that are customers of the business.

4.2.1 Client Schema

Field	Type	Required	Notes / Constraints
id	UUID / Int PK	Yes	Auto-generated primary key
firstName	String	Yes	Max 100 chars
lastName	String	Yes	Max 100 chars
email	String	Yes	Unique. Max 255 chars.
company	String	No	Optional. Max 200 chars.
socialMedia	JSON / JSONB	No	Key-value map: { instagram, twitter, linkedin, tiktok, ... }
userId	UUID / Int FK	Yes	References User.id — the admin owner of this client record.
subscribed	Boolean	Yes	Default: false. Marketing / communication opt-in.

Field	Type	Required	Notes / Constraints
taxId	String	No	Optional. EIN, VAT, or SSN. Must be encrypted at rest.
measurements	JSON / JSONB	No	Flexible measurement fields. Must include a "unit" key ("in" or "cm").
createdAt	DateTime	Yes	Auto-set on creation
updatedAt	DateTime	Yes	Auto-set on update

4.2.2 Client Business Rules

- Each client must be linked to exactly one User (userId required).
- taxId must be encrypted at rest; only decrypted for admin-role API responses.
- Deleting a client cascades: deletes all Projects (and their Tasks and ProjectInvoice records). Invoice records themselves are preserved.

4.3 Project

Represents a unit of billable work assigned to a client. Projects link to tasks, project types, and invoices via join tables.

4.3.1 Project Schema

Field	Type	Required	Notes / Constraints
id	UUID / Int PK	Yes	Auto-generated primary key
name	String	Yes	Display title. Max 300 chars.
description	Rich Text	No	Optional. Rendered in UI.
status	Enum	Yes	Draft In Progress Completed On Hold Cancelled. Default: Draft.
notes	Rich Text	No	Admin-only internal notes. Never returned in client-role responses.
dueDate	Date	No	Target completion date. ISO 8601.
price	Decimal(10,2)	No	Total agreed project price. Must be ≥ 0 .
paid	Boolean	Yes	Default: false.
completed	Boolean	Yes	Default: false. Syncs with status = Completed.
clientId	UUID / Int FK	Yes	References Client.id

Field	Type	Required	Notes / Constraints
<code>userId</code>	UUID / Int FK	Yes	References User.id — the admin owner
<code>createdAt</code>	DateTime	Yes	Auto-set on creation
<code>updatedAt</code>	DateTime	Yes	Auto-set on update

4.3.2 Project Business Rules

- Setting status = Completed auto-sets completed = true, and vice versa.
- notes must not appear in any client-role API response.
- Deleting a project cascades: deletes all Tasks and ProjectInvoice join records. Invoice records are NOT deleted.
- Deleting a project also removes its ProjectProjectType join records.

4.4 Invoice

First-class billing entity. An invoice belongs to a client and may be associated with one or more projects via the ProjectInvoice join table. Contains line items and tracks its own payment lifecycle.

✓ **New in v1.2:** Invoice.projectId has been removed. Use the ProjectInvoice join table to link invoices to one or more projects.

4.4.1 Invoice Schema

Field	Type	Required	Notes / Constraints
<code>id</code>	UUID / Int PK	Yes	Auto-generated primary key
<code>invoiceNumber</code>	String	Yes	Unique. Human-readable. e.g. INV-2024-001.
<code>clientId</code>	UUID / Int FK	Yes	References Client.id. Who is being billed.
<code>status</code>	Enum	Yes	Draft Sent Paid Overdue Cancelled. Default: Draft.
<code>issueDate</code>	Date	Yes	Date the invoice was or will be issued. ISO 8601.
<code>dueDate</code>	Date	No	Payment due date. Past this date unpaid = Overdue.

Field	Type	Required	Notes / Constraints
paidAt	DateTime	No	Required when status = Paid.
subtotal	Decimal(10,2)	Yes	Sum of all line item totals. Recomputed on line item changes.
taxRate	Decimal(5,4)	No	e.g. 0.0825 = 8.25%. Default: 0.
taxAmount	Decimal(10,2)	Yes	Computed: subtotal * taxRate. Default: 0.
total	Decimal(10,2)	Yes	Computed: subtotal + taxAmount.
notes	Rich Text	No	Internal or client-facing payment terms.
userId	UUID / Int FK	Yes	References User.id — the admin who created this invoice.
createdAt	DateTime	Yes	Auto-set on creation
updatedAt	DateTime	Yes	Auto-set on update

4.4.2 Invoice Status Lifecycle

Status	Description
Draft	Created but not sent. Freely editable.
Sent	Delivered to client. Line items are locked from further changes.
Paid	Payment received. paidAt must be set.
Overdue	Past dueDate and still unpaid. Set manually or by a scheduled job.
Cancelled	Voided. No payment expected. Preserved for audit.

4.4.3 Invoice Business Rules

- invoiceNumber must be unique. Suggested auto-format: INV-YYYY-### per year.
- subtotal, taxAmount, and total must be recomputed whenever a line item is saved or deleted.
- Line items are locked from changes when status is Sent, Paid, Overdue, or Cancelled.
- paidAt is required when setting status = Paid.
- Hard delete is only permitted for Draft and Cancelled invoices. Sent and beyond must use status = Cancelled.
- An invoice can be linked to zero, one, or many projects via the ProjectInvoice join table.

4.5 ProjectInvoice Join Table

Implements the Many-to-Many relationship between Project and Invoice. Replaces the single Invoice.projectId FK from v1.1.0, enabling bundled invoices that span multiple projects.

✓ **New in v1.2:** Real-world example: a client commissions a wedding dress (sewing), wedding invitations (design), and a wedding website (web). One invoice — "Wedding Package" — links to all three projects via this join table.

4.5.1 ProjectInvoice Schema

Field	Type	Required	Notes / Constraints
projectId	UUID / Int FK	Yes	References Project.id. Part of composite PK.
invoiceId	UUID / Int FK	Yes	References Invoice.id. Part of composite PK.
linkedAt	DateTime	No	Timestamp when this project was linked to the invoice.

4.5.2 ProjectInvoice Business Rules

- Composite PK (projectId, invoiceId) prevents duplicate links.
- Deleting a Project cascades to remove its ProjectInvoice records. The Invoice itself is preserved.
- Deleting an Invoice cascades to remove its ProjectInvoice records. The Projects are preserved.
- A project may appear on multiple invoices (e.g. a deposit invoice + a final invoice for the same project).

4.6 InvoiceLineItem

Individual lines on an invoice. May reference a Service template or be freeform.

4.6.1 InvoiceLineItem Schema

Field	Type	Required	Notes / Constraints
id	UUID / Int PK	Yes	Auto-generated primary key

Field	Type	Required	Notes / Constraints
<code>invoiceId</code>	UUID / Int FK	Yes	References Invoice.id
<code>serviceId</code>	UUID / Int FK	No	Optional. References Service.id. Pre-fills description and unitPrice.
<code>description</code>	String	Yes	Line item label shown on invoice. Max 500 chars.
<code>quantity</code>	Decimal(10,2)	Yes	Default: 1. Supports fractional quantities (e.g. 1.5 hours).
<code>unitPrice</code>	Decimal(10,2)	Yes	Price per unit. Must be ≥ 0 .
<code>lineTotal</code>	Decimal(10,2)	Yes	Computed: $\text{quantity} * \text{unitPrice}$. Stored for auditability.
<code>sortOrder</code>	Integer	No	Display order within the invoice.
<code>createdAt</code>	DateTime	Yes	Auto-set on creation

4.6.2 InvoiceLineItem Business Rules

- $\text{lineTotal} = \text{quantity} * \text{unitPrice}$. Always recompute on save.
- Adding, updating, or deleting a line item triggers recomputation of Invoice.subtotal, taxAmount, and total.
- Line items cannot be modified when the parent Invoice status is Sent, Paid, Overdue, or Cancelled.

4.7 Service

Reusable catalog of billable services. Acts as a template for invoice line items.

4.7.1 Service Schema

Field	Type	Required	Notes / Constraints
<code>id</code>	UUID / Int PK	Yes	Auto-generated primary key
<code>name</code>	String	Yes	Unique. Max 200 chars. e.g. "Logo Design", "Suit Alterations".
<code>description</code>	Rich Text	No	What the service includes.
<code>defaultPrice</code>	Decimal(10,2)	No	Optional suggested price pre-filled on line item creation.
<code>category</code>	Enum	No	Website Design Sewing Other. Used for filtering.

Field	Type	Required	Notes / Constraints
<code>active</code>	Boolean	Yes	Default: true. Inactive services are hidden from line item selection.
<code>createdAt</code>	DateTime	Yes	Auto-set on creation
<code>updatedAt</code>	DateTime	Yes	Auto-set on update

4.7.2 Service Business Rules

- name must be unique within the Service table.
- Deactivating a service does not affect existing InvoiceLineItems that reference it.
- Hard delete is blocked if any InvoiceLineItem references the service. Use active = false instead.

4.8 Task

Individual actionable items scoped to a project. Supports one level of subtasks via a self-referencing `parentTaskId` field.

✓ **New in v1.2:** `parentTaskId` is the new field in v1.2. A null value = root/parent task. A non-null value = subtask. Maximum one level of nesting is enforced at the API layer — a subtask cannot itself have children.

4.8.1 Task Schema

Field	Type	Required	Notes / Constraints
<code>id</code>	UUID / Int PK	Yes	Auto-generated primary key
<code>projectId</code>	UUID / Int FK	Yes	References Project.id. All tasks (including subtasks) belong to a project.
<code>parentTaskId</code>	UUID / Int FK	No	Self-referencing. Null = root task. Non-null = subtask of <code>parentTaskId</code> .
<code>title</code>	String	Yes	Short task label. Max 300 chars.
<code>description</code>	Rich Text	No	Optional detailed description or acceptance criteria.
<code>status</code>	Enum	Yes	To Do In Progress Blocked Done Cancelled. Default: To Do.

Field	Type	Required	Notes / Constraints
priority	Enum	No	Low Medium High Urgent. Default: Medium.
dueDate	Date	No	Optional deadline. ISO 8601.
completedAt	DateTime	No	Auto-set when status transitions to Done.
assignedTo	UUID / Int FK	No	Optional. References User.id (admin-role users only).
sortOrder	Integer	No	Display order within the project or parent task.
createdAt	DateTime	Yes	Auto-set on creation
updatedAt	DateTime	Yes	Auto-set on update

4.8.2 Task Business Rules

- A task must always belong to a project (projectId required), including subtasks.
- A subtask (parentTaskId set) must belong to the same project as its parent.
- Maximum nesting depth is 1. A task that already has a parentTaskId cannot itself be a parent. Enforce at the API layer.
- Setting status = Done auto-sets completedAt to the current timestamp.
- Deleting a parent task cascades to delete all of its subtasks.
- Deleting a project cascades to delete all tasks and subtasks belonging to it.
- assignedTo may only reference admin-role users.

4.9 ProjectType

Defines the category of work a project involves. Hierarchical: top-level category, optional sewing subcategory, optional pattern style detail link for Custom sewing work.

4.9.1 ProjectType Schema

Field	Type	Required	Notes / Constraints
id	UUID / Int PK	Yes	Auto-generated primary key
name	String	Yes	e.g. "Logo Design", "Wedding Dress". Max 200 chars.
category	Enum	Yes	Website Design Sewing
sewingSubcategory	Enum	Cond.	Required when category = Sewing. Values: Tailoring Bespoke Custom.

Field	Type	Required	Notes / Constraints
<code>description</code>	String	No	Optional. Max 500 chars.
<code>createdAt</code>	DateTime	Yes	Auto-set on creation

4.9.2 ProjectType Examples

Name	Category	Sewing Subcategory	Has Pattern Style?
Web Development	Website	—	No
Logo Design	Design	—	No
Branding Package	Design	—	No
Suit Alterations	Sewing	Tailoring	No
Custom Suit	Sewing	Bespoke	No
Wedding Dress	Sewing	Custom	Yes
Custom Gown	Sewing	Custom	Yes

4.10 PatternStyleDetail

4.10.1 PatternStyleDetail Schema

Field	Type	Required	Notes / Constraints
<code>id</code>	UUID / Int PK	Yes	Auto-generated primary key
<code>patternStyleNumber</code>	String	Yes	e.g. "Vogue V9123". Max 100 chars.
<code>patternStyleTypeId</code>	UUID / Int FK	Yes	References PatternStyleType.id
<code>projectId</code>	UUID / Int FK	Yes	References ProjectType.id. Unique — one detail per project type.
<code>createdAt</code>	DateTime	Yes	Auto-set on creation

4.10.2 PatternStyleDetail Business Rules

- `projectId` is unique — only one PatternStyleDetail per ProjectType.
- Only valid when the linked ProjectType has `sewingSubcategory = Custom`.

4.11 PatternStyleType

4.11.1 PatternStyleType Schema

Field	Type	Required	Notes / Constraints
id	UUID / Int PK	Yes	Auto-generated primary key
name	String	Yes	Unique. e.g. "Vogue", "McCall's". Max 200 chars.
notes	Rich Text	No	Admin notes about this pattern type.
muslinMade	Boolean	Yes	Default: false.
dateMuslinMade	Date	Cond.	Required when muslinMade = true. ISO 8601.
createdAt	DateTime	Yes	Auto-set on creation
updatedAt	DateTime	Yes	Auto-set on update

5. Entity Relationships

Complete relationship map for all 11 entities and 2 join tables.

From Entity	Relation	To Entity	Details
User	One-to-Many →	AuthCode	A user may have multiple AuthCode records over time. FK: AuthCode.userId.
User	One-to-Many →	Client	A user (admin) owns many clients. FK: Client.userId.
User	One-to-Many →	Project	A user (admin) owns many projects. FK: Project.userId.
User	One-to-Many →	Invoice	A user (admin) creates invoices. FK: Invoice.userId.
User	One-to-Many →	Task	A user may be assigned to many tasks. FK: Task.assignedTo.
Client	One-to-Many →	Project	A client can have many projects. FK: Project.clientId.
Client	One-to-Many →	Invoice	A client can have many invoices. FK: Invoice.clientId.
Project	Many-to-Many ↔	ProjectType	Via ProjectProjectType join table.
Project	Many-to-Many ↔	Invoice	Via ProjectInvoice join table. NEW in v1.2.
Project	One-to-Many →	Task	A project has many tasks (root and subtasks). FK: Task.projectId.
Task	Self One-to-Many →	Task (subtasks)	parentTaskId self-reference. Max depth 1. NEW in v1.2.
Invoice	One-to-Many →	InvoiceLineItem	FK: InvoiceLineItem.invoiceId. Cascades on delete.
InvoiceLineItem	Many-to-One →	Service	Optional. FK: InvoiceLineItem.serviceId.
ProjectType	One-to-One →	PatternStyleDetail	Custom sewing types only. FK: PatternStyleDetail.projectTypeId (unique).
PatternStyleDetail	Many-to-One →	PatternStyleType	FK: PatternStyleDetail.patternStyleTypeId.

5.1 Join Table: ProjectProjectType

ProjectProjectType Schema

Field	Type	Required	Notes / Constraints
projectId	UUID / Int FK	Yes	References Project.id. Composite PK.
projectTypeId	UUID / Int FK	Yes	References ProjectType.id. Composite PK.
assignedAt	DateTime	No	When the type was assigned to the project.

5.2 Join Table: ProjectInvoice

ProjectInvoice Schema

Field	Type	Required	Notes / Constraints
projectId	UUID / Int FK	Yes	References Project.id. Composite PK.
invoiceId	UUID / Int FK	Yes	References Invoice.id. Composite PK.
linkedAt	DateTime	No	When this project was linked to the invoice.

6. API Requirements

6.1 Auth

Method	Endpoint	Role	Description
POST	/auth/request-code	Public	Submit email to receive login code. Rate-limited 5/email/10 min.

Method	Endpoint	Role	Description
POST	/auth/verify-code	Public	Submit email + code. Returns JWT + sets refresh cookie.
POST	/auth/refresh	Cookie	Exchange refresh token for new access token. Rotates token.
DELETE	/auth/logout	Authenticated	Invalidate refresh token and clear cookie.

6.2 Users

Method	Endpoint	Role	Description
GET	/users	Admin	List all non-deleted users
GET	/users/:id	Admin	Get single user
PATCH	/users/:id	Admin	Update user (role, email)
DELETE	/users/:id	Admin	Soft-delete user

6.3 Clients

Method	Endpoint	Role	Description
GET	/clients	Admin	List all clients. Filter by subscribed, search by name/email.
POST	/clients	Admin	Create a new client
GET	/clients/:id	Admin/Owner	Get client. taxId masked unless admin explicitly requests reveal.
PATCH	/clients/:id	Admin	Update client
DELETE	/clients/:id	Admin	Delete client. Cascades to projects and tasks.

6.4 Projects

Method	Endpoint	Role	Description
GET	/projects	Admin	List all projects. Filter by status, client, type, due date.
POST	/projects	Admin	Create project. Requires at least one projectTypeid.
GET	/projects/:id	Admin/Owner	Get project. notes hidden from client-role.
PATCH	/projects/:id	Admin	Update project. Status syncs completed field.
DELETE	/projects/:id	Admin	Delete project. Cascades tasks + join records.
GET	/projects/:id/tasks	Admin/Owner	List root tasks and their subtasks for a project.
GET	/projects/:id/invoices	Admin	List invoices linked to a project via ProjectInvoice.

6.5 Invoices

Method	Endpoint	Role	Description
GET	/invoices	Admin	List all invoices. Filter by status, client, date range.
POST	/invoices	Admin	Create invoice (Draft). Optionally pass projectIds[] to link via ProjectInvoice.
GET	/invoices/:id	Admin	Get invoice with line items and linked projects.
PATCH	/invoices/:id	Admin	Update invoice metadata. Locked when Sent+.
DELETE	/invoices/:id	Admin	Hard delete only if Draft or Cancelled.
POST	/invoices/:id/projects	Admin	Link a project to this invoice (add ProjectInvoice record).
DELETE	/invoices/:id/projects/:pid	Admin	Unlink a project from this invoice.
POST	/invoices/:id/line-items	Admin	Add line item. Triggers total recompute. Locked when Sent+.

Method	Endpoint	Role	Description
PATCH	/invoices/:id/line-items/:liId	Admin	Update line item. Locked when Sent+.
DELETE	/invoices/:id/line-items/:liId	Admin	Remove line item. Triggers recompute. Locked when Sent+.
POST	/invoices/:id/send	Admin	Transition Draft → Sent. Locks line items.
POST	/invoices/:id/mark-paid	Admin	Transition → Paid. Sets paidAt.

6.6 Tasks

Method	Endpoint	Role	Description
GET	/tasks	Admin	List all tasks. Filter by status, project, assignee.
POST	/tasks	Admin	Create task or subtask. If parentTaskId provided, validates max depth.
GET	/tasks/:id	Admin	Get task with its subtasks.
PATCH	/tasks/:id	Admin	Update task. status = Done auto-sets completedAt.
DELETE	/tasks/:id	Admin	Delete task. Cascades to subtasks.
GET	/tasks/:id/subtasks	Admin	List subtasks for a given parent task.

6.7 Services

Method	Endpoint	Role	Description
GET	/services	Authenticated	List active services. Admin sees all incl. inactive.
POST	/services	Admin	Create service
GET	/services/:id	Authenticated	Get single service

Method	Endpoint	Role	Description
PATCH	/services/:id	Admin	Update. active = false to deactivate.
DELETE	/services/:id	Admin	Block delete if referenced by any line item. Return 409.

6.8 Project Types & Pattern Styles

Method	Endpoint	Role	Description
GET	/project-types	Authenticated	List all project types
POST	/project-types	Admin	Create. Validates sewing subcategory rules.
PATCH	/project-types/:id	Admin	Update project type
DELETE	/project-types/:id	Admin	Block delete if referenced by projects. Return 409.
GET	/pattern-style-types	Authenticated	List pattern style types
POST	/pattern-style-types	Admin	Create. Validates dateMuslinMade when muslinMade = true.
PATCH	/pattern-style-types/:id	Admin	Update
GET	/pattern-style-details	Admin	List all pattern style details
POST	/pattern-style-details	Admin	Create. Only valid for Custom sewing project types.
PATCH	/pattern-style-details/:id	Admin	Update pattern style detail

7. Business Logic & Validation Rules

7.1 Conditional Validation Matrix

Condition	Requirement	HTTP Response if Violated
ProjectType.category = Sewing	sewingSubcategory must be provided	422 Unprocessable Entity
ProjectType.category != Sewing	sewingSubcategory must be null	422 Unprocessable Entity
sewingSubcategory = Custom	PatternStyleDetail may be linked (optional)	N/A
sewingSubcategory != Custom	PatternStyleDetail must NOT be linked	422 Unprocessable Entity
PatternStyleType.muslinMade = true	dateMuslinMade must be provided	422 Unprocessable Entity
Project.status = Completed	completed auto-set to true	Auto-coerced, no error
Project.completed = true	status auto-set to Completed	Auto-coerced, no error
Invoice.status = Paid	paidAt must be set	422 if missing on manual set
Invoice.status = Sent/Paid/Overdue	Line items locked from changes	409 Conflict
InvoiceLineItem saved	lineTotal = qty * unitPrice recomputed	Internal enforcement
InvoiceLineItem deleted	Invoice subtotal/tax/total recomputed	Internal enforcement
Task.status = Done	completedAt auto-set to now	Auto-coerced, no error
Task has parentTaskId set	parent must have parentTaskId = null (max depth)	422 Unprocessable Entity
Task subtask created	projectId must match parent task projectId	422 Unprocessable Entity
AuthCode: new code requested	Invalidate any prior active code for user	Internal enforcement
AuthCode.attempts >= 5	Invalidate code immediately	429 Too Many Requests
Service deleted	Block if referenced by InvoiceLineItem	409 Conflict
ProjectType deleted	Block if referenced by any project	409 Conflict

7.2 Cascade & Delete Behavior

Entity Deleted	Cascade Behavior
User	Soft delete only (sets deletedAt). All related data preserved. User cannot authenticate.
Client	Hard cascade: deletes all Projects (and their Tasks, ProjectProjectType, ProjectInvoice records). Invoice records preserved.
Project	Hard cascade: deletes Tasks (+ subtasks), ProjectProjectType records, ProjectInvoice records. Invoices preserved.
Task (root)	Hard cascade: deletes all subtasks of this task.
Task (subtask)	Hard delete. No further cascade.
Invoice (Draft/Cancelled)	Hard delete. Cascades to InvoiceLineItems and ProjectInvoice records.
Invoice (Sent+)	BLOCKED. Set status = Cancelled instead.
InvoiceLineItem	Hard delete. Triggers Invoice total recomputation.
Service	BLOCKED if referenced by any InvoiceLineItem. Use active = false.
ProjectType	BLOCKED if referenced by any project via join table. Return 409.
PatternStyleType	BLOCKED if any PatternStyleDetail references it. Return 409.

8. Security Considerations

- All traffic must use HTTPS/TLS. Redirect HTTP to HTTPS in production.
- Auth codes: short-lived (15 min), single-use, hashed at rest. Raw code never logged or stored.
- Access tokens stored in JavaScript memory only — never localStorage or sessionStorage.
- Refresh tokens travel only via httpOnly, Secure, SameSite=Strict cookies.
- Refresh token reuse detection: revoked token submitted → invalidate ALL sessions for that user immediately.
- Rate-limit /auth/request-code: 5 requests per email per 10 minutes.
- RBAC enforced server-side on every request. Client-side guards are UI convenience only.
- taxId encrypted at rest (AES-256). Decrypt only for authorized admin API responses.
- All user input sanitised before DB writes — prevent SQLi, XSS, prototype pollution.
- Production API responses must never expose stack traces or internal error messages.
- CORS must use an explicit origin allowlist. No wildcard (*) in production.
- All auth events (request-code, verify-code, refresh, logout, failed attempts) must be logged with IP, userId, and timestamp. Retain logs for 90 days minimum.

9. Frontend Requirements

9.1 Auth UI

- Step 1: Email input form with "Send code" button.
- Step 2: 6-digit code input with countdown timer (15 min). Resend link disabled for 60 s after send.
- On success: redirect to dashboard. Token stored in memory only.

9.2 Dashboard Views

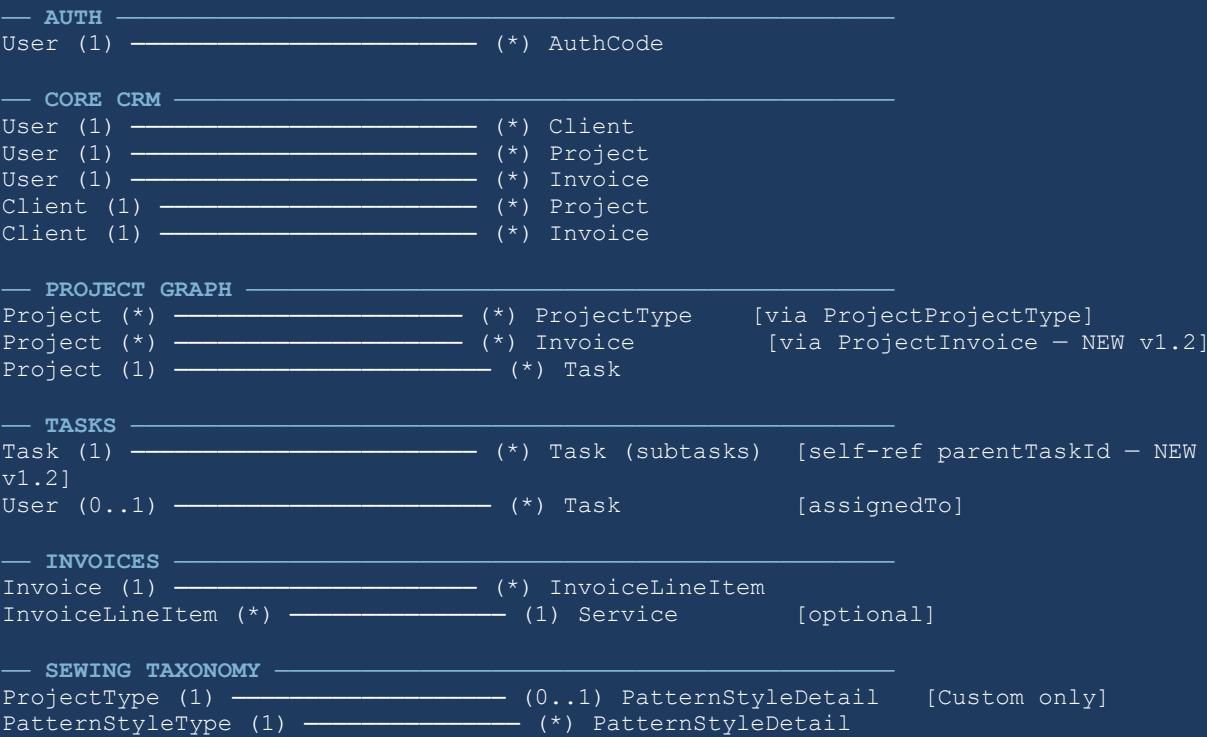
- Admin Overview — summary cards: total clients, active projects, open invoices, overdue invoices, unpaid revenue total.
- Client List — searchable, filterable by subscribed status.
- Client Detail — profile, measurements accordion, linked projects, invoice history.
- Project List — filter by status, category, client, due date range.
- Project Detail — info panel, task board (root tasks + subtasks), assigned types, linked invoices, pattern details if applicable.
- Invoice List — filter by status, client, date range. Overdue invoices highlighted.
- Invoice Detail — line items table with totals, project links, status action buttons (Send, Mark Paid).
- Invoice Create/Edit — project selector (multi-select from ProjectInvoice), service catalog picker for line items.
- Task Board — Kanban view. Root tasks as cards, subtasks as nested items within each card.
- Service Catalog — CRUD table with active/inactive toggle.
- Project Type Manager — CRUD. sewingSubcategory and Pattern Style fields shown conditionally.

9.3 Role-Based UI Rules

- Client-role: read-only access to own profile, projects (no notes), and own invoices. No pricing on project cards.
- Admin-role: full read/write. taxId masked (last 4 only) unless explicitly revealed.
- sewingSubcategory fields only visible when category = Sewing.
- Pattern Style fields only visible when sewingSubcategory = Custom.
- Invoice line item controls disabled when status is Sent or beyond.
- Task "Add subtask" button only visible on root-level tasks (parentTaskId = null).

10. Entity Relationship Summary

Full Entity Relationship Diagram (v1.2.0)



11. Revision History

Version	Date	Author	Summary
1.0.0	—	Project Team	Initial draft. Core entities: User, Client, Project, ProjectType, PatternStyleDetail, PatternStyleType.

Version	Date	Author	Summary
1.1.0	—	Project Team	Added Invoice, InvoiceLineItem, Task, Service. Passwordless auth (AuthCode entity). Removed password field from User.
1.2.0	4/9/2026	Project Team	Replaced Invoice.projectId with ProjectInvoice M:M join table. Added Task.parentTaskId for one-level subtask support.